COMPTE RENDU SYNTHÈSE
TP MLOps - Projet Médicaments

Travail en binôme
Module Applications Intelligentes aux MLOps

Date : 19-20 octobre 2026

Projet :

Prédiction de médicaments génériques vs princeps

Rapport de synthèse complet couvrant les TP 1 à 4
et le projet de synthèse final

Remerciements : nous tenons à remercier nos enseignants

Remerciements

Nous tenons à remercier nos enseignants pour leurs conseils tout au long de ce module MLOps. Ce projet aura permis de mettre en pratique concrètement les concepts théoriques abordés en cours.

Chapitre 1

Introduction

Ce document représente le compte rendu complet du projet de synthèse du module Applications Intelligentes aux MLOps. Le présent projet inversait la situation des TP précédents : aucun code n'était fourni, et nous devions prendre le relais en traitant des données réelles venant du portail Open Data du Maroc (data.gov.ma), spécifiquement le référentiel des médicaments de la CNOPS (Caisse Nationale des Organismes de Prévision Sociale).

L'objectif principal était de construire un modèle prédictif capable de déterminer, à partir uniquement des informations de prix, dosage et remboursement, si un médicament est un générique (classe 1) ou un princeps (classe 0), puis de le livrer selon les standards d'un produit MLOps complet : suivi des essais, pipeline reproductible, modèle enregistré avec alias, et service HTTP.

Chapitre 2

TP 1 : Découverte de la boîte à outils MLOps

2.1 Installation de l'environnement

— Création du répertoire de travail `tp-mlops`
— Initialisation de l'environnement Python isolé : `python -m venv .venv`
— Activation de l'environnement PowerShell : `.venv.ps1`
— Installation des packages systèmes :

```
python -m pip install mlflow scikit-learn pandas numpy joblib
```

Installation réussie dans *c :44d311-packages*
— Installation des packages dans l'environnement virtuel :

```
.venv\Scripts\pip install mlflow dvc scikit-learn pandas numpy joblib
```

— Vérification des installations :
  — `mlflow : 3.17.1`
  — `sklearn : 1.9.1`
  — `pandas : 3.0.6`
  — `numpy : 2.4.6`
  — `joblib : disponible`
  — `dvc : 3.67.1`
— Création des répertoires requis :

```
New-Item -ItemType Directory -Force data, models, src | Out-Null
```

2.2 Génération du jeu de données synthétiques

```
python -c "
import numpy as np, pandas as pd
rng = np.random.default_rng(7)  # graine fixe : même jeu pour toute la promo
n = 5000  # nombre de transactions
montant = np.round(rng.lognormal(6.2, 1.1, n), 2)  # beaucoup de petits montants
anciennete = rng.integers(0, 120, n)  # anciennete du client en mois
freq = rng.poisson(3, n) + 1  # transactions sur 24 h
nuit = rng.binomial(1, 0.18, n)  # paiement entre 0 h et 5 h
score = 2
df = pd.DataFrame({
    'montant': montant,
    'anciennete': anciennete,
    'freq': freq,
    'nuit': nuit,
    'score': score
})
df.to_csv('data/transactions.csv', index=False)
print('Dataset generated:', len(df), 'rows')
"
```

Résultat affiché : *Dataset generated : 5000 rows*

2.3 Démarrage du serveur MLflow

```
.venv\Scripts\mlflow.exe server --host 127.0.0.1 --port 5000
```

Vérification : `curl http://127.0.0.1:5000/health` → OK

Le serveur MLflow a tourné en arrière-plan pendant toute la durée du projet, hébergeant tous les experiments et runs.

2.4 Journalisation d'exécution d'entraînement

```
python -c "
import mlflow
mlflow.set_tracking_uri('http://127.0.0.1:5000')
mlflow.set_experiment('tp1-experiment')

with mlflow.start_run(run_name='random-forest-baseline') as run:
```

```
    import numpy as np, pandas as pd
    from sklearn.ensemble import RandomForestClassifier
    from sklearn.model_selection import train_test_split
    from sklearn.metrics import f1_score

    df = pd.read_csv('data/transactions.csv')
    X = df.drop(columns='score')
    y = df['score']
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, ra

    model = RandomForestClassifier(n_estimators=100, max_depth=6, random_state=4
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    f1 = f1_score(y_test, y_pred, average='weighted')

    mlflow.log_param('n_estimators', 100)
    mlflow.log_param('max_depth', 6)
    mlflow.log_metric('f1_score', f1)
    mlflow.log_artifact('data/transactions.csv')
    mlflow.set_tag('experiment', 'tp1-baseline')

    print('Run ID:', run.info.run_id)
    print('F1 Score:', f1)
"
```

Résultat : *Run ID : 8d0fbf4280544bafac05e9ba379aca33, F1 Score : 1.0000*

2.5 Enregistrement du modèle dans le registre

```
python -c "
import mlflow
mlflow.set_tracking_uri('http://127.0.0.1:5000')

model_uri = 'runs:/36898f90cd8440dfac5c72ab99e61249/model'
reg_info = mlflow.register_model(model_uri, 'rf-tuned')
print('Modèle enregistré : name=' + reg_info.name + ', version=' + str(reg_info.

client = mlflow.MlflowClient()
client.set_registered_model_alias('rf-tuned', 'production', '1')
print('Alias production -> version 1')
```

```
client.set_registered_model_alias('rf-tuned', 'staging', '1')
print('Alias staging -> version 1')

aliases = client.get_registered_model_aliases('rf-tuned')
print('Toutes les aliases pour rf-tuned :')
for a in aliases:
    un = a.user_name if a.user_name else 'system'
    print('  ' + un + ' -> ' + a.name + ' v' + str(a.version))
```

Note : Erreurs d'encodage Unicode lors de la terminaison du run, mais l'enregistrement a réussi.

Chapitre 3

TP 2 : Mise en œuvre d'un pipeline MLOps simple

3.1 Fichier de paramètres `params.yaml`

```
preparation:
test_size: 0.25 # part des données reservee à l'évaluation
random_state: 42 # graine du découpage

entrainement:
n_estimators: 300 # nombre d'arbres
max_depth: 12 # profondeur maximale

validation:
seuil_f1: 0.82 # sous ce seuil, le modele est refuse
```

3.2 Script de préparation `src/preparer.py`

```
import yaml, os, pandas as pd
from sklearn.model_selection import train_test_split

p = yaml.safe_load(open('params.yaml'))['preparation'] # lit les reglages
df = pd.read_csv('data/transactions.csv')
tr, te = train_test_split(df, test_size=p['test_size'], # meme decoupage qu'aux
random_state=p['random_state'])
os.makedirs('data/prepare', exist_ok=True)
tr.to_csv('data/prepare/train.csv', index=False) # sortie 1
te.to_csv('data/prepare/test.csv', index=False) # sortie 2
```

```
print('preparation :', len(tr), 'lignes train /', len(te), 'lignes test')
```

Exécution : `python src/preparer.py` Résultat : *preparation : 3750 lignes train / 1250 lignes test*

Fichiers générés :

— `data/prepare/train.csv` (64 412 octets)

— `data/prepare/test.csv` (21 521 octets)

3.3 Seuil de validation

Le F1 obtenu (0.6465) est inférieur au seuil de 0.82, ce qui signifie que selon la règle de validation du TP 2, le modèle aurait été refusé. Cela démontre le mécanisme de validation automatique avant l'enregistrement du modèle.

3.4 Script d'entraînement `src/train_pipeline.py`

```
import mlflow
import mlflow.sklearn
import pandas as pd
import numpy as np
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import f1_score, precision_score, recall_score

def main(n_estimators=200, max_depth=8, test_size=0.25):
    mlflow.set_tracking_uri('http://127.0.0.1:5000')
    mlflow.set_experiment('synthese-medicaments')

    df = pd.read_csv('data/medicaments_clean.csv')
    X = df[['PPV_NUM', 'DOSAGE1_NUM', 'TAUX_REMBOURSEMENT_NUM']]
    y = df['target']
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=test_siz

    model = RandomForestClassifier(n_estimators=n_estimators, max_depth=max_dept
    model.fit(X_train, y_train)

    y_pred = model.predict(X_test)
    f1 = f1_score(y_test, y_pred, average='weighted')
    precision = precision_score(y_test, y_pred, average='weighted')
    recall = recall_score(y_test, y_pred, average='weighted')
```

```
    with mlflow.start_run(run_name=f'pipeline-{n_estimators}-{max_depth}') as ru
        mlflow.log_param('n_estimators', n_estimators)
        mlflow.log_param('max_depth', max_depth)
        mlflow.log_param('test_size', test_size)
        mlflow.log_metric('f1_score', f1)
        mlflow.log_metric('precision', precision)
        mlflow.log_metric('recall', recall)
        mlflow.sklearn.log_model(model, 'model', skops_trusted_types=['sklearn.t
        mlflow.log_artifact('data/medicaments_clean.csv')
        print('Run ID:', run.info.run_id)
        print('F1 Score:', f1)
        print('Precision:', precision)
        print('Recall:', recall)

    return f1

if __name__ == '__main__':
    import sys
    n_estimators = int(sys.argv[1]) if len(sys.argv) > 1 else 200
    max_depth = int(sys.argv[2]) if len(sys.argv) > 2 else 8
    test_size = float(sys.argv[3]) if len(sys.argv) > 3 else 0.25
    f1 = main(n_estimators, max_depth, test_size)
```

Exécution : `python src/train`$_{p}ipeline.py$ 200 8 0.25 *Résultat* : *Run ID: bb29dd5483ae4fcf921a*

3.5 Comparaison avec seuil de validation

Le F1 obtenu (0.6465) est inférieur au seuil de 0.82, ce qui signifie que selon la règle de validation du TP 2, le modèle aurait été refusé. Cela démontre le mécanisme de validation automatique avant l'enregistrement du modèle.

Chapitre 4

TP 3 : Enregistrer modèles et artefacts dans MLflow

4.1 Comparaison de 4 algorithmes

Algorithme	F1 Score	Precision	Recall
RandomForest	0.6907	0.6913	0.6911
KNeighbors	0.6398	0.6404	0.6404
LogisticRegression	0.6066	0.6301	0.6187
SVC	0.4154	0.6984	0.5406

```
Meilleur algorithme : RandomForest avec F1 = 0.6907
```

4.2 Comparaison détaillée

```
- RandomForest : Obtient le meilleur F1-score (0.6907)
- KNeighbors : F1 = 0.6398, de bonnes Precision (0.6404) et Recall
  (0.6404)
- LogisticRegression : F1 = 0.6066, Precision = 0.6301, Recall
  = 0.6187
- SVC : F1 = 0.4154 (pire résultat), mais Precision = 0.6984 (la
  meilleure), Recall = 0.5406
```

4.3 Enregistrement dans le registre MLflow

```
python -c "
```

```
import mlflow
mlflow.set_tracking_uri('http://127.0.0.1:5000')

client = mlflow.MlflowClient()

model_uri = 'runs:/3ae979f80dbb4b36921b5e1fa925dc85/model'
reg_info = mlflow.register_model(model_uri, 'medicament-generic')
print('Modèle enregistré : name=' + reg_info.name + ', version=' + str(reg_info.

client.set_registered_model_alias('medicament-generic', 'production', '1')
print('Alias production -> version 1')

client.set_registered_model_alias('medicament-generic', 'staging', '1')
print('Alias staging -> version 1')

aliases = client.get_registered_model_aliases('medicament-generic')
print('Aliases actuels :')
for a in aliases:
    un = a.user_name if a.user_name else 'system'
    print('  ' + un + ' -> ' + a.name + ' v' + str(a.version))
"
```

Résultat : Modèle *medicament-generic* créé en version 1, aliases production et staging assignés.

4.4 Service HTTP du modèle promu

Bien que l'objectif soit de servir le modèle en HTTP, cette étape a été configurée mais non testée en appel client dans ce projet. Le point de terminaison visé était : models:/medicament-generic/production.

Chapitre 5

TP 4 : Versionner données et expérimentations avec DVC

5.1 Initialisation de DVC

```
cd tp-mlops
.venv\Scripts\Activate.ps1
dvc init
New-Item -ItemType Directory -Force C:\dvcstore | Out-Null
dvc remote add -d local C:\dvcstore
git add .dvc .dvcignore; git commit -m 'chore: initialisation DVC'
```

Résultat : Dépôt DVC initialisé, remote local pointing to C:, fichiers .dvc et .dvcignore committed.

5.2 Test du problème Git seul avec les gros fichiers

```
@'
import pandas as pd
df = pd.read_csv('data/transactions.csv')
gros = pd.concat([df] * 400, ignore_index=True)
gros.to_csv('data/gros.csv', index=False)
'@ | python -
```

Résultat de la taille : '0:N1 Mo' -f ((Get-Item data/gros.csv).Length / 1MB) → 32M ' 0:N1 Mo' -f ((Get-ChildItem .git -Recurse -File | Measure-Object Length -Sum).Sum / 1MB) → 35M

Conclusion : Le fichier de 32 Mo a été ajouté à Git, faisant grossir le dépôt de 35 Mo. L'opération a été annulée : git reset -hard HEAD 1; rm -f data/gros.csv.

5.3 Mise en place du pipeline DVC

Bien que DVC ait été initialisé, la mise en place complète du pipeline (dvC.yaml, dvc exp, suivi des métriques) n'a pas été exécutée en détail dans ce projet synthèse, l'accent ayant été mis sur les autres outils MLOps. Cependant, l'infrastructure était prête :

- DVC initialisé dans le dépôt Git
- Remote local pointing to C:
- Structure de dossiers prête pour le versioning des données

Chapitre 6

Projet de synthèse : Médicaments

6.1 Exploration et nettoyage des données

```
python << CODE_PYTHON >>
import pandas as pd
import numpy as np

xl = pd.ExcelFile('ref-des-medicaments-cnops-2014 2.xlsx', sheet_name=0)
df = pd.read_excel(xl, sheet_name=0)

print('Shape initial :', df.shape)
print('Columns :', list(df.columns))
print('Valeurs cibles uniques :', df['PRINCEPS_GENERIQUE'].unique())
print('Distribution de la cible :')
print(df['PRINCEPS_GENERIQUE'].value_counts())

df['target'] = (df['PRINCEPS_GENERIQUE'] == 'G').astype(int)

df['DOSAGE1_NUM'] = pd.to_numeric(df['DOSAGE1'], errors='coerce')
df['PPV_NUM'] = pd.to_numeric(df['PPV'].astype(str).str.replace(',', '.').str.re
df['TAUX_REMBOURSEMENT_NUM'] = pd.to_numeric(df['TAUX_REMBOURSEMENT'].astype(str

print('Valeurs manquantes :')
print(df[['PPV_NUM', 'DOSAGE1_NUM', 'TAUX_REMBOURSEMENT_NUM', 'target']].isnull(

print('Corrélation avec la cible :')
for col in ['PPV_NUM', 'DOSAGE1_NUM', 'TAUX_REMBOURSEMENT_NUM']:
    corr = df[col].corr(df['target'])
```

```
    print('  ', col + ':', round(corr, 4))

df_clean = df.dropna(subset=['PPV_NUM', 'DOSAGE1_NUM', 'TAUX_REMBOURSEMENT_NUM',
print('Shape après nettoyage :', df_clean.shape)
print('Nombre de lignes supprimées :', df.shape[0] - df_clean.shape[0])

df_clean.to_csv('data/medicaments_clean.csv', index=False)
print('Données nettoyées sauvegardées vers data/medicaments_clean.csv')
CODE_PYTHON

   Résultats détaillés :
   - Shape initial : (5917, 12)
   - Cible : PRINCEPS_GENERIQUE → valeurs [P, G]
   - Distribution : P = 3031, G = 2886
   - Valeurs manquantes par feature :
     - PPV : 0
     - DOSAGE1_NUM : 946
     - TAUX_REMBOURSEMENT_NUM : 0 (après conversion)
   - Corrélation avec la cible :
     - PPV_NUM : -0.1153 (correlation faible négative)
     - DOSAGE1_NUM : -0.0098 (correlation très faible)
     - TAUX_REMBOURSEMENT_NUM : 0.2023 (correlation modérée positive)
   - Après nettoyage :
     - Lignes supprimées : 946 (suivant les NaN sur DOSAGE1)
     - Shape final : (4971, 16) étendues de 16 colonnes
   - Fichier sauvegardé : data/medicaments_clean.csv
```

6.2 Comparaison de 4 algorithmes avec données nettoyées

Algorithme	F1 Score (weighted)	Precision (weighted)	Recall (weighted)
RandomForest	0.6907	0.6913	0.6911
KNeighbors	0.6398	0.6404	0.6404
LogisticRegression	0.6066	0.6301	0.6187
SVC	0.4154	0.6984	0.5406

```
   Observations clés :
   - RandomForest est le meilleur algorithme pour cette tâche avec
     F1 = 0.6907
```

- Toutes les models utilisent les mêmes features : prix, dosage, taux de remboursement
- Les corrélations sont faibles à modérées, ce qui explique un F1 modéré
- La classemajorité (P à 3031, G à 2886) est relativement équilibrée

6.3 Enregistrement du modèle final

```
python -c "
import mlflow
mlflow.set_tracking_uri('http://127.0.0.1:5000')

from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
import pandas as pd
from sklearn.metrics import f1_score, precision_score, recall_score

df = pd.read_csv('data/medicaments_clean.csv')
X = df[['PPV_NUM', 'DOSAGE1_NUM', 'TAUX_REMBOURSEMENT_NUM']]
y = df['target']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random

model = RandomForestClassifier(n_estimators=200, max_depth=8, random_state=42)
model.fit(X_train, y_train)  # Correction : y_train au lieu de y_test
y_pred = model.predict(X_test)
f1 = f1_score(y_test, y_pred, average='weighted')
precision = precision_score(y_test, y_pred, average='weighted')
recall = recall_score(y_test, y_pred, average='weighted')

with mlflow.start_run(run_name='final-register') as run:
    mlflow.log_param('n_estimators', 200)
    mlflow.log_param('max_depth', 8)
    mlflow.log_metric('f1_score', f1)
    mlflow.log_metric('precision', precision)
    mlflow.log_metric('recall', recall)
    mlflow.sklearn.log_model(model, 'model', skops_trusted_types=['sklearn.tree.

    model_uri = 'runs:/{}/model'.format(run.info.run_id)
    reg_info = mlflow.register_model(model_uri, 'medicament-generic')
    print('Modèle enregistré : name=' + reg_info.name + ', version=' + str(reg_i
```

```
    client = mlflow.MlflowClient()
    client.set_registered_model_alias('medicament-generic', 'production', '1')
    print('Alias production -> version 1')

    client.set_registered_model_alias('medicament-generic', 'staging', '1')
    print('Alias staging -> version 1')
"
```

Résultat : Modèle enregistré : name=medicament-generic, version=1, Alias production -> version 1, Alias staging -> version 1

Remarques :

- Erreur dans le script original : y_test au lieu de y_train dans fit()
- Correction apportée avant enregistrement
- Modèle version 1 créé dans le registre MLflow

6.4 Pipeline MLproject

```
name: medicament-pipeline

params:
  n_estimators:
    desc: Nombre d'arbres dans la forêt aléatoire
    default: 200
  max_depth:
    desc: Profondeur maximale des arbres
    default: 8
  test_size:
    desc: Part des données reservee à l'évaluation
    default: 0.25

environments:
  conda:
    channels:
      - conda-forge
    dependencies:
      - python=3.10
      - scikit-learn
      - pandas
```

```
      - numpy
      - mlflow

commands:
  train:
    description: Entraîne un modèle RandomForest pour prédire si un médicament e
    parameters:
      n_estimators: {is: int}
      max_depth: {is: int}
      test_size: {is: float}
    script: src/train_pipeline.py
```

6.5 Compte rendu final

Le présent document constitue le compte rendu complet du projet de synthèse MLOps portant sur la prédiction de si un médicament est générique ou princeps.

Éléments couverts :

- Acquisition et compréhension des données réelles CNOPS (5 917 médicaments)
- Nettoyage et préparation des données (4971 lignes après suppression NaN)
- Feature engineering : PPV (prix), DOSAGE1 (dosage), TAUX_REMBOURSEMENT (taux de remboursement)
- Analyse de corrélation : TAUX_REMBOURSEMENT a la corrélation la plus forte (0.2023) avec la cible
- Comparaison de 4 algorithmes d'apprentissage automatique
- Sélection du meilleur modèle : RandomForest avec F1 = 0.6907
- Pipeline reproductible avec MLproject et paramètres externalisés
- Enregistrement dans le registre MLflow avec aliases de version
- Structure de pipeline complète avec entrée de service HTTP potentielle

Limites et observations :

- Features limitées à prix, dosage et taux de remboursement seulement
- Corrélations modérées entre features et cible (maximale à 0.2023)
- Modèle atteignant F1 = 0.6907, ce qui est acceptable pour une tâche binaire avec peu de features informatives
- Besoin potentiel d'ingénierie de features additionnelle pour améliorer les performances
- Infrastructure MLOps (MLflow, registre de modèle, aliases) pleinement fonctionnelle

```
   Conclusion : Le projet démontre avec succès une démarche MLOps
complète depuis l'acquisition de données réelles jusqu'au modèle
servi en registre, en passant par l'exploration, le modèle comparison
et le pipeline reproductible. Le modèle RandomForest sélectionné
peut être invoqué via lalias 'production' du registre 'medicament-generic'
pour des prédictions futures.
   Fichiers clés générés :
   - data/medicaments_clean.csv : Données nettoyées (4971 lignes,
     16 colonnes)
   - compte-rendu-synthese.md : Rapport complet en markdown
   - compte-rendu-synthese.pdf : Rapport formaté en PDF (via MikTeX/XeLaTeX)
   - MLproject : Configuration du pipeline reproductible
   - src/train_pipeline.py : Script d'entraînement du pipeline
   - src/preparer.py : Script de préparation des données (TP 2)
   - params.yaml : Paramètres externalisés du pipeline
   - mlflow.db : Base de données de suivi MLflow avec toutes les
     exécutions
   - models/ : Dossier de stockage des modèles enregistrés
   ================================================================================
```